

H.W#5

Q1: Multidimensional arrays can be stored in row major order, as in C++, or in column major order, as in Fortran. Develop the access functions for both arrangements for 3-dimensional arrays. You can start your answer w/:

Let the lower bounds of subscript ranges of the three dimensions be named $\min(1)$, $\min(2)$ & $\min(3)$, respectively. Let the sizes of the subscript ranges be $\text{size}(1)$, $\text{size}(2)$, & $\text{size}(3)$ respectively. Assume the element size is n .

given a matrix of $[2][3][4]$

$\text{size}(1) = \text{size of } i = 2$

$\text{size}(2) = \text{size of } j = 3$

$\text{size}(3) = \text{size of } k = 4$

row-major order:

0, 1, 2, 3, ..., 20, 21, 22, 23

col-major order:

k			
0	1	2	3
4	5	6	7
8	9	10	11
k			
12	13	14	15
16	17	18	19
20	21	22	23

maybe I can say, given the dimension 1 offset can be represented by i
 dimension 2 offset can be represented by j
 dimension 3 offset can be represented by k .

and the $\text{size}(1) = \text{number of valid indices in dimension 1}$

$\text{size}(2) = \text{number of valid indices in dimension 2}$

$\text{size}(3) = \text{number of valid indices in dimension 3}$

the total offset can be represented by:

$$\text{offset} := i \cdot \text{size}(2) \cdot \text{size}(3) + j \cdot \text{size}(3) + k$$

and to access this element, we can do something like:

$$\text{element} := \text{base-address} + \text{size-of-element} \cdot \text{offset}$$

another way to think about it is by starting w/

$$\text{offset} = k + j \cdot \text{size}(z) + i \cdot \text{size}(z) \cdot \text{size}(x)$$

$$\text{factor} = k + \text{size}(z) [j + i \cdot \text{size}(x)]$$

In general, for any d-dimensional array
 $N_1 \times N_2 \times \dots \times N_d$

to access the element:

$$[n_1][n_2][n_3] \dots [n_d]$$

and assuming all indices are zero-based, then the offset can be computed w/

$$\text{offset} := N_d + N_d(n_{d-1} + N_{d-1}(\dots \frac{1}{N_2}(n_2 + N_2 \cdot n_1) \dots))$$

and accessing the element w/:

$$\text{element} := \text{base_address} + \text{size_of_element} \cdot \text{offset}$$

Column major:

column major assumes the index of the left increases the fastest. Suppose we have a $2 \times 3 \times 4$ matrix, then

$A[0][0][0]$ = first element

$A[1][0][0]$ = second element

$A[0][1][0]$ = third element

$A[1][1][0]$ = fourth element

$A[0][2][0]$ = fifth

⋮

$A[0][1][3]$ = 21st element

$A[1][1][3]$ = 22nd element

$A[0][2][3]$ = 23rd element

$A[1][2][3]$ = last element

to calculate the offset in column-major, use:

$$\text{offset} = n_1 + N_1 (n_2 + N_2 (\dots + N_{d-2} (n_{d-1} + N_{d-1} \cdot n_d) \dots))$$

Accessing the element remains the same, so long as you generate the offset correctly:

$$\text{element} := \text{base-address} + \text{size-of-element} \cdot \text{offset}$$

Q2

What are the arguments for & against Java's implicit heap storage recovery, when compared w/ the explicit heap storage recovery required in C++ (consider real time systems).

Pro Java: 1) Automatically reclaim unreachable objects. This reduces memory leaks.

2) Prevents dangling pointers because Java prevents explicitly releasing memory.

3) It generally makes it safer to use the heap.

Pro C++: 1) The JVM is an interpreter and requires runtime overhead.

2) Freeing memory is unpredictable, in Java. C++ gives you explicit control, making it more predictable.

3)

Q3

Define static type checking & dynamic type checking. Give two reasons why static type checking is better than dynamic type checking.

• Static type checking means typing is verified at compile time, while

• dynamic type checking verifies typing at runtime.

• Two benefits of static type checking is

1) type errors are caught earlier

2) run-time checking is more expensive than static checking.
(dynamic)

Q4

Explain how coercion rules can weaken the beneficial effect of strong typing.

= coercion results in more chances that you do not detect a type error. It can make implicit castings harder to detect, which would not occur in a more strongly typed language.

Q5

Assume the following rules of associativity & precedence for expressions:

* , / , not
+ , - , & , mod
- (unary)
= , != , < , <= , >= , >
and
or , xor

Show the order of evaluation for the following.

- a) $a * b - 1 + c \rightarrow ((a * b) - 1)^2 + c)^3$
 b) $a * (b - 1) / c \text{ mod } d \rightarrow ((a * (b - 1))^2 / c)^3 \text{ mod } d)^4$
 c) $(a - b) / c \& (d * e / a - 3) \rightarrow (((a - b) / c)^5 \& (((d * e)^2 / a)^3 - 3)^4)^6$
 d) $-a \text{ or } c = d \text{ and } e \rightarrow ((-a) \text{ or } ((c = d)^2 \text{ and } e)^3)^4$
 e) $a > b \text{ xor } c \text{ or } d <= 17 \rightarrow (((a > b) \text{ xor } c)^3 \text{ or } (d <= 17)^2)^4$
 f) $-a + b \rightarrow -(a + b)^2$

Q6

write a BNF description of the precedence and associativity rules defined for the expressions in Problem 5. Assume the only operands are the names a, b, c, d, & e

$\{expr\} \rightarrow \{expr\} \{expr'\} \mid \{and\}$
 $\{expr'\} \rightarrow \text{or } \{and\} \mid \text{xor } \{and\}$
 $\{and\} \rightarrow \{comparisons\} \{and'\}$
 $\{and'\} \rightarrow \text{and } \{comparisons\} \{and'\} \mid \epsilon$

$\{expr\} \rightarrow \{and\} \{expr'\}$
 $\{expr'\} \rightarrow \text{or } \{and\} \{expr'\} \mid \text{xor } \{and\} \{expr'\} \mid \epsilon$
 $\{and\} \rightarrow \{comparisons\} \{and'\}$
 $\{and'\} \rightarrow \text{and } \{comparisons\} \{and'\} \mid \epsilon$
 $\{comparisons\} \rightarrow \{negate\} \{comparisons'\}$
 $\{comparisons'\} \rightarrow = \{neg\} \{c'\} \mid ! = \{neg\} \{c'\} \mid < \{neg\} \{c'\} \mid <= \{neg\} \{c'\} \mid > \{neg\} \{c'\} \mid >= \{neg\} \{c'\} \mid \epsilon$

$\{negate\} \rightarrow - \{add\} \mid \{add\}$
 $\{add\} \rightarrow \{multiply\} \{add'\}$
 $\{add'\} \rightarrow + \{m\} \{add'\} \mid - \{m\} \{add'\} \mid \& \{m\} \{add'\} \mid \text{mod } \{multiply\} \{add'\} \mid \epsilon$
 $\{multiply\} \rightarrow \{operands\} \{m'\} \mid \{operands\}$

$$\{m'\} \rightarrow * \{operands\} \{m'\} / \{operands\} \{m'\} \in$$

$$\{operands\} \rightarrow a \mid b \mid c \mid d \mid e$$

Q7. Can the distributive law of boolean Algebra fail under the effect of short-circuit evaluation & side-effectful expression? If your answer is "yes", you should support your argument w/ a specific example.

$$1) (B_1 \&\& (B_2 \parallel B_3)) \equiv ((B_1 \&\& B_2) \parallel (B_1 \&\& B_3))$$

$$2) (B_1 \parallel (B_2 \&\& B_3)) \equiv ((B_1 \parallel B_2) \&\& (B_1 \parallel B_3))$$

$$1) B_1 = T \rightarrow F ; B_2 = F ; B_3 = T$$

$$T \&\& (F \parallel T) \equiv (T \&\& F) \parallel (T \&\& T)$$

$$B_1 = X^{++} < 5, \text{ where } X = 4$$

$$B_2 = F$$

$$B_3 = T$$

$$2) B_1 = T \rightarrow F, B_2 = F, B_3 = F$$

$$(T \parallel (F \&\& F)) \equiv ((T \parallel F) \&\& (F \parallel F))$$

$$B_1 = X^{++} < 5, \text{ where } X = 4$$

$$B_2 = F$$

$$B_3 = F$$

Q8

What is the difference between operational semantics & denotational semantics?

Operational semantics describe how a program executes by specifying the sequence of state changes, until the program finishes executing.

Denotational semantics is a formal method for describing the meaning of programs. It does this by mapping syntactic objects of the language to mathematical objects that represent their meaning. In other words, denotational semantics map the syntactic domain, consisting of syntactic objects, to the semantic domain, which consist of math objects. There are many mapping functions, each defining how to interpret a syntactic object.

Q9 using the virtual machine instructions given in section 3.5.1.1, give an operational semantic definition of the following:

a) Java do-while

b) C for

a) $\text{do } \{ \text{while}(\text{expr1}); \text{expr2}; \}$ $\rightarrow$ $\text{loop: } \text{expr2}; // \text{update loop}$
 $\text{if}(\text{expr1}) \text{ goto loop} // \text{if } \text{expr1} == 1 \text{ goto loop}$
 else goto out
 $\text{out: } \dots$

b) $\text{for}(\text{expr1}; \text{expr2}; \text{expr3}) \{ \dots \}$ $\rightarrow$ $\text{loop: } \text{if}(\text{expr2} == 0) \text{ goto out}$
 $\text{expr3};$
 goto loop
 $\text{out: } \dots$

Q10 write denotational semantics mapping function for the following constructs. Examples of such mapping functions can be found in Sebastia p. 411 & 412

a) Java do-while

b) Java for

a) $M_{DL}(\text{do } L \text{ while } B, S) \triangleq$
 $\text{if } M_{SL}(L, S) == \text{error}$
 then error;
 else
 $S' = M_{SL}(L, S);$ $\rightarrow \text{if } M_B(B, S') == \text{undef then error}$
 $\text{if } M_B(B, S') == \text{false;}$
 $\text{then } S';$
 else
 $M_{DL}(\text{do } L \text{ while } B, S')$